



Target: CruShead1999

Eugenio D. Deluca (KarasuR00T)

Date: 14/08/2024

Hey there! Well, the purpose of this tutorial is to try and solve this old crackme that taught many of us back when we were starting out.

Specifically, Ricardo Narvaja showed how to solve it using Olly in his old tutorials—in INTRODUCTION TO CRACKING WITH OLLYDBG PART 16. Although he shows how to reach a solution, my goal is to approach it again after all this time and create a keygen so it works with any name.

I'm going to stick with Olly—obviously, the path is identical to Ricardo's original tutorial up to a certain point.

Alright, first I'm going to set a BP on GetDlgItemTextA—this is because, according to MSDN, it 'retrieves the title or text associated with a control in a dialog box.'

OllyDbg - CRACKME.EXE

File View Debug Plugins Options Window Help

LEMTWHC / KBR... S

CPU - main thread, module CRACKME

Address	Hex dump	ASCII
00401000	00 00 00 00 00 00 00 00	
00401001	00 00 00 00 00 00 00 00	
00401002	00 00 00 00 00 00 00 00	
00401003	00 00 00 00 00 00 00 00	
00401004	00 00 00 00 00 00 00 00	
00401005	00 00 00 00 00 00 00 00	
00401006	00 00 00 00 00 00 00 00	
00401007	00 00 00 00 00 00 00 00	
00401008	00 00 00 00 00 00 00 00	
00401009	00 00 00 00 00 00 00 00	
0040100A	00 00 00 00 00 00 00 00	
0040100B	00 00 00 00 00 00 00 00	
0040100C	00 00 00 00 00 00 00 00	
0040100D	00 00 00 00 00 00 00 00	
0040100E	00 00 00 00 00 00 00 00	
0040100F	00 00 00 00 00 00 00 00	
00401010	00 00 00 00 00 00 00 00	
00401011	00 00 00 00 00 00 00 00	
00401012	00 00 00 00 00 00 00 00	
00401013	00 00 00 00 00 00 00 00	
00401014	00 00 00 00 00 00 00 00	
00401015	00 00 00 00 00 00 00 00	
00401016	00 00 00 00 00 00 00 00	
00401017	00 00 00 00 00 00 00 00	
00401018	00 00 00 00 00 00 00 00	
00401019	00 00 00 00 00 00 00 00	
0040101A	00 00 00 00 00 00 00 00	
0040101B	00 00 00 00 00 00 00 00	
0040101C	00 00 00 00 00 00 00 00	
0040101D	00 00 00 00 00 00 00 00	
0040101E	00 00 00 00 00 00 00 00	
0040101F	00 00 00 00 00 00 00 00	
00401020	00 00 00 00 00 00 00 00	
00401021	00 00 00 00 00 00 00 00	
00401022	00 00 00 00 00 00 00 00	
00401023	00 00 00 00 00 00 00 00	
00401024	00 00 00 00 00 00 00 00	
00401025	00 00 00 00 00 00 00 00	
00401026	00 00 00 00 00 00 00 00	
00401027	00 00 00 00 00 00 00 00	
00401028	00 00 00 00 00 00 00 00	
00401029	00 00 00 00 00 00 00 00	
0040102A	00 00 00 00 00 00 00 00	
0040102B	00 00 00 00 00 00 00 00	
0040102C	00 00 00 00 00 00 00 00	
0040102D	00 00 00 00 00 00 00 00	
0040102E	00 00 00 00 00 00 00 00	
0040102F	00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00	
00401031	00 00 00 00 00 00 00 00	
00401032	00 00 00 00 00 00 00 00	
00401033	00 00 00 00 00 00 00 00	
00401034	00 00 00 00 00 00 00 00	
00401035	00 00 00 00 00 00 00 00	
00401036	00 00 00 00 00 00 00 00	
00401037	00 00 00 00 00 00 00 00	
00401038	00 00 00 00 00 00 00 00	
00401039	00 00 00 00 00 00 00 00	
0040103A	00 00 00 00 00 00 00 00	
0040103B	00 00 00 00 00 00 00 00	
0040103C	00 00 00 00 00 00 00 00	
0040103D	00 00 00 00 00 00 00 00	
0040103E	00 00 00 00 00 00 00 00	
0040103F	00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00	
00401041	00 00 00 00 00 00 00 00	
00401042	00 00 00 00 00 00 00 00	
00401043	00 00 00 00 00 00 00 00	
00401044	00 00 00 00 00 00 00 00	
00401045	00 00 00 00 00 00 00 00	
00401046	00 00 00 00 00 00 00 00	
00401047	00 00 00 00 00 00 00 00	
00401048	00 00 00 00 00 00 00 00	
00401049	00 00 00 00 00 00 00 00	
0040104A	00 00 00 00 00 00 00 00	
0040104B	00 00 00 00 00 00 00 00	
0040104C	00 00 00 00 00 00 00 00	
0040104D	00 00 00 00 00 00 00 00	
0040104E	00 00 00 00 00 00 00 00	
0040104F	00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00	
00401051	00 00 00 00 00 00 00 00	
00401052	00 00 00 00 00 00 00 00	
00401053	00 00 00 00 00 00 00 00	
00401054	00 00 00 00 00 00 00 00	
00401055	00 00 00 00 00 00 00 00	
00401056	00 00 00 00 00 00 00 00	
00401057	00 00 00 00 00 00 00 00	
00401058	00 00 00 00 00 00 00 00	
00401059	00 00 00 00 00 00 00 00	
0040105A	00 00 00 00 00 00 00 00	
0040105B	00 00 00 00 00 00 00 00	
0040105C	00 00 00 00 00 00 00 00	
0040105D	00 00 00 00 00 00 00 00	
0040105E	00 00 00 00 00 00 00 00	
0040105F	00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00	
00401061	00 00 00 00 00 00 00 00	
00401062	00 00 00 00 00 00 00 00	
00401063	00 00 00 00 00 00 00 00	
00401064	00 00 00 00 00 00 00 00	
00401065	00 00 00 00 00 00 00 00	
00401066	00 00 00 00 00 00 00 00	
00401067	00 00 00 00 00 00 00 00	
00401068	00 00 00 00 00 00 00 00	
00401069	00 00 00 00 00 00 00 00	
0040106A	00 00 00 00 00 00 00 00	
0040106B	00 00 00 00 00 00 00 00	
0040106C	00 00 00 00 00 00 00 00	
0040106D	00 00 00 00 00 00 00 00	

Registers (FPU)

Register	Value
EAX	00000000
ECX	0012FFB0
EDX	7C91E4F4 ntdll.KernelBase.dll
EBX	7FFD0000
ESP	0012FFC4
EBP	0012FFFD
ESI	FFFFFFFF
EDI	7C920208 ntdll.KernelBase.dll
EIP	00401000 CRACKME.00401000
EAX	00000000 (NO, NO, E, ...)
ECX	empty -UNORM BB00
EDX	empty 0.0
EBX	empty 0.0
ESP	empty 0.0
EBP	empty 0.0
ESI	empty 0.0
EDI	empty 0.0
EIP	empty 1.0000000000
ST0	empty 1.0000000000
ST1	empty 1.0000000000
ST2	empty 1.0000000000
ST3	empty 1.0000000000
ST4	empty 1.0000000000
ST5	empty 1.0000000000
ST6	empty 1.0000000000
ST7	empty 1.0000000000
FST	4020 Cond 1 0 0 0
PCW	027F Prec NEAR, 53

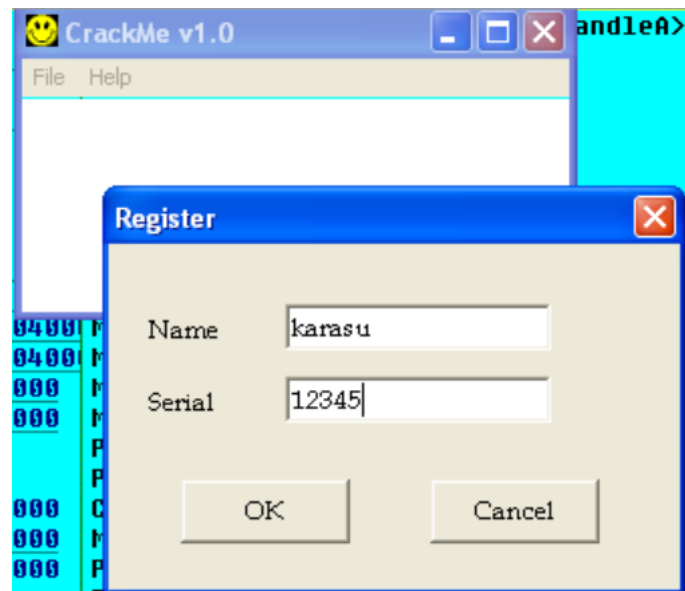
ASCII

Address	Hex dump	ASCII
0012FFC4	00 00 00 00 00 00 00 00	
0012FFC5	00 00 00 00 00 00 00 00	
0012FFC6	00 00 00 00 00 00 00 00	
0012FFC7	00 00 00 00 00 00 00 00	
0012FFC8	00 00 00 00 00 00 00 00	
0012FFC9	00 00 00 00 00 00 00 00	
0012FFCA	00 00 00 00 00 00 00 00	
0012FFCB	00 00 00 00 00 00 00 00	
0012FFCC	00 00 00 00 00 00 00 00	
0012FFCD	00 00 00 00 00 00 00 00	
0012FFCE	00 00 00 00 00 00 00 00	
0012FFCF	00 00 00 00 00 00 00 00	
0012FFD0	00 00 00 00 00 00 00 00	
0012FFD1	00 00 00 00 00 00 00 00	
0012FFD2	00 00 00 00 00 00 00 00	
0012FFD3	00 00 00 00 00 00 00 00	
0012FFD4	00 00 00 00 00 00 00 00	

Command: BP GetDlgItemTextA BP address, string - Break with condition

Program entry point

We hit RUN and get to the corresponding window -> if we click on the 'Help' -> 'Register' option, the famous registration screen appears, asking us for a name and a serial.



So I'll start testing—I'll use 'karasu' as the name and '12345' as the serial, and then I'll hit OK. It stops at the MOV EDI,EDI instruction -> but the important thing here is what appears on the STACK -> that's where the input for the Name is stored— meaning 'karasu'. Alright, let's move on -> Right-click -> Follow in Dump. If I look at the dump window, it's at 0—this is because, as Ricardo explains, it hasn't written anything yet; the API hasn't executed yet. We hit Ctrl + F9, or go to the top menu -> Debug -> Execute till return.

7E3EB084	74 00	JE SHORT USER32.7E3EB08C
7E3EB086	8B45 10	MOV EAX,DWORD PTR SS:[EBP+10]
7E3EB089	C600 00	MOV BYTE PTR DS:[EAX],0
7E3EB08C	33C0	XOR EAX,EAX
7E3EB08E	5D	POP EBP
7E3EB08F	C2 1000	RETN 10
7E3EB092	90	NOP
7E3EB093	90	NOP
7E3EB094	90	NOP
7E3EB095	90	NOP
7E3EB096	90	NOP
7E3EB097	8BFF	MOV EDI,EDI

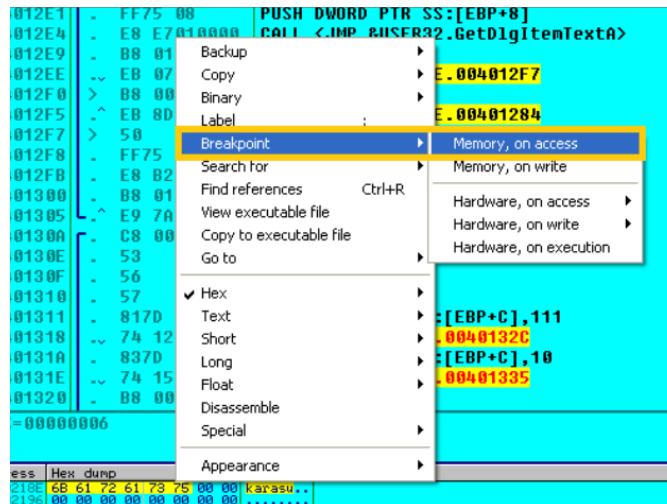
Return to 004012C9 (CRACKME.004012C9)

Address	Hex dump	ASCII
0040218E	68 61 72 61 73 75 00 00	karasu..
00402196	00 00 00 00 00 00 00 00	
0040219E	00 00 00 00 00 00 00 00	
004021A6	00 00 00 00 00 00 00 00	
004021AE	00 00 00 00 00 00 00 00	
004021B6	00 00 00 00 00 00 00 00	
004021BE	00 00 00 00 00 00 00 00	
004021C6	00 00 00 00 00 00 00 00	

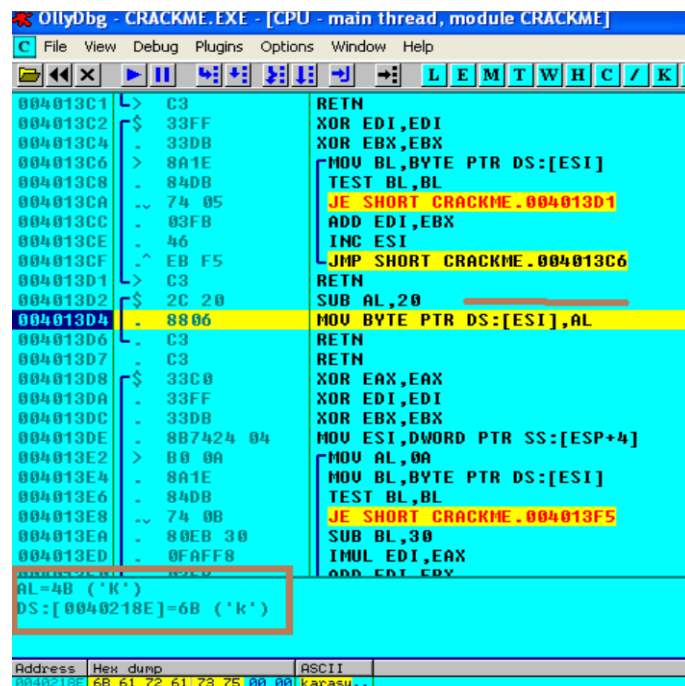
Command BP GetDlgItemTextA

From this point on, I'm going to deviate a bit from what was shown in Ricardo's original tutorial.

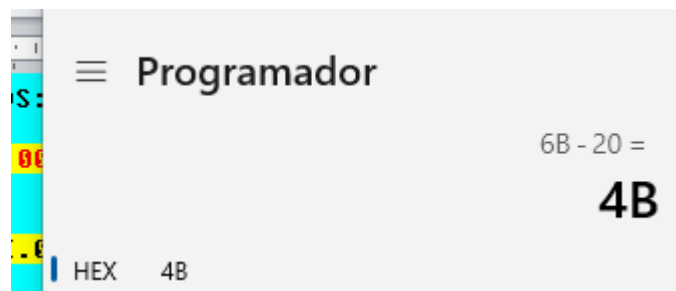
Now we can see that it wrote the name—OK -> F7—now what I'm going to do is highlight the hexadecimal value in the dump—I'll mark 6B 61 72 61 73 75—Then I'm going to set a Breakpoint on Access -> to do this -> right-click ->



And we hit F9 or RUN (x2, meaning twice, to reach the instruction) -> Let's remember that it stops one instruction before—meaning that MOV AL, BYTE PTR DS:[EDI] hasn't executed yet -> if we look at Olly's comments, it indicates that [004018E] holds 6B (which is the hex value for 'k' in the ASCII table) and it gets saved into AL— I keep tracing and reach this point, which I believe is important:



Let's remember that AL = 6B -> then it subtracts 20 in SUB AL, 20 -> so this means that AL is now equal to 4B



And that means that 'k' is now 4B (which is 'K' in ASCII) and then there is a RETURN (RET) -> so it goes back to -> Return to 00401399 (CRACKME.00401399)

As we keep tracing -> I hit the loop again -> notice that now it takes the 'a' and AL retains the value 4B ('K')

Address	Disassembly
00401383	MOV AL, BYTE PTR DS:[ESI]
00401385	TEST AL, AL
00401387	JE SHORT CRACKME.0040139C
00401389	CMP AL, 41
0040138B	JB SHORT CRACKME.004013AC
0040138D	CMP AL, 5A
0040138F	JNB SHORT CRACKME.00401394
00401391	INC ESI
00401392	JMP SHORT CRACKME.00401383
00401394	CALL CRACKME.004013D2
00401399	INC ESI
0040139A	JMP SHORT CRACKME.00401383
0040139C	POP ESI
0040139D	CALL CRACKME.004013C2
004013A2	XOR EDI, 5678
004013A8	MOV EAX, EDI
004013AA	JMP SHORT CRACKME.004013C1
004013AC	POP ESI
004013AD	PUSH 30
004013AF	PUSH CRACKME.00402160
004013B4	PUSH CRACKME.00402169
004013B9	PUSH DWORD PTR SS:[EBP+8]
004013BC	CALL <JMP.&USER32.MessageBoxA>
004013C1	RETN
004013C2	XOR EDI, EDI

DS:[0040218F]=61 ('a')

AL=4B ('K')

Jumps from 00401392, 0040139A

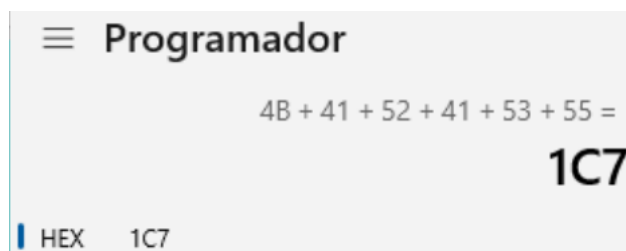
In the next instruction AL = 61, I keep tracing and once again:

004013D2	\$ 2C 28	SUB AL,20
004013D4	- 8806	MOV BYTE PTR DS:[ESI],AL
004013D6	- C3	RETN
004013D7	- C3	RETN
004013D8	\$ 33C0	XOR EAX,EAX
004013DA	- 33FF	XOR EDI,EDI
004013DC	- 33DB	XOR EBX,EBX
004013DE	- 8B7424 04	MOV ESI,DWORD PTR SS:[ESP+4]
004013E2	> B0 0A	MOV AL,0A
004013E4	- 8A1E	MOV BL,BYTE PTR DS:[ESI]
004013E6	- 84DB	TEST BL,BL
004013E8	~ 74 0B	JE SHORT CRACKME.004013F5
004013EA	- 80EB 30	SUB BL,30
004013ED	- 0FAFF8	IMUL EDI,EAX
004013F0	- 03FB	ADD EDI,EBX
004013F2	- 46	INC ESI
004013F3	^ EB ED	JMP SHORT CRACKME.004013E2
004013F5	> 81F7 3412000	XOR EDI,1234
004013F8	- 8BDF	MOV EBX,EDI
004013FD	- C3	RETN
004013FE	- FF25 8431400	JMP DWORD PTR DS:[&USER32.KillTimer]
00401404	- FF25 8831400	JMP DWORD PTR DS:[&USER32.GetSystemMet
0040140A	\$- FF25 8C31400	JMP DWORD PTR DS:[&USER32.LoadCursorA]
00401410	- FF25 9031400	JMP DWORD PTR DS:[&USER32.LoadAccelerat
00401416	\$- FF25 9431400	JMP DWORD PTR DS:[&USER32.MessageBeep]
AL=41 ('A')		
DS:[0040218F]=61 ('a')		

In other words, it takes the next letter again and converts it to uppercase -> and then it keeps gathering the result in EDI— tokeep it from getting tedious, what it does is loop through the string entered in the name -> puts it all in uppercase -> and then adds up each character—so

if I entered 'karasu' -> it will basically convert it to 'KARASU' and then add up each character -> KARASU = 4B 41 52 41 53 55 (in Hex)

Then it adds it up character by character



Once it loops through the entire string and exits the loop, we arrive here:

Address	Disassembly	Comment
00401394	> E8 39000000	CALL CRACKME.004013D2
00401399	. 46	INC ESI
0040139A	^ EB E7	JMP SHORT CRACKME.00401383
0040139C	> 5E	POP ESI
0040139D	. E8 20000000	CALL CRACKME.004013C2
004013A2	. 81F7 78560000	XOR EDI,5678
004013A8	. 8BC7	MOV EAX,EDI
004013AA	~ EB 15	JMP SHORT CRACKME.004013C1
004013AC	> 5E	POP ESI
004013AD	. 6A 30	PUSH 30
004013AF	. 68 60214000	PUSH CRACKME.00402160
004013B4	. 68 69214000	PUSH CRACKME.00402169
004013B9	. FF75 08	PUSH DWORD PTR SS:[EBP+8]
004013BC	. E8 79000000	CALL <JMP.&USER32.MessageBoxA>
004013C1	> C3	RETN
004013C2	\$ 33FF	XOR EDI,EDI
004013C4	. 33DB	XOR EBX,EBX
004013C6	> 8A1E	MOV BL, BYTE PTR DS:[ESI]
004013C8	. 84DB	TEST BL,BL
004013CA	~ 74 05	JE SHORT CRACKME.004013D1
004013CC	. 03FB	ADD EDI,EBX
004013CE	. 46	INC ESI
004013CF	^ EB F5	JMP SHORT CRACKME.004013C6
004013D1	> C3	RETN
004013D2	^ 2C 20	SUB AL, 20

Style = MB_OK
Title = "No 1"
Text = "No 1"
hOwner
MessageBoxA

EDI=000001C7

Notice that, sure enough, EDI = 1C7-> Then it does a XOR EDI, 5678 = 57BF and subsequently moves that value into EAX ->

Address	Disassembly	Comment
004013A8	. 8BC7	MOV EAX,EDI
004013AA	~ EB 15	JMP SHORT CRACKME.004013C1
004013AC	> 5E	POP ESI
004013AD	. 6A 30	PUSH 30
004013AF	. 68 60214000	PUSH CRACKME.00402160
004013B4	. 68 69214000	PUSH CRACKME.00402169
004013B9	. FF75 08	PUSH DWORD PTR SS:[EBP+8]
004013BC	. E8 79000000	CALL <JMP.&USER32.MessageBoxA>
004013C1	> C3	RETN
004013C2	\$ 33FF	XOR EDI,EDI
004013C4	. 33DB	XOR EBX,EBX
004013C6	> 8A1E	MOV BL, BYTE PTR DS:[ESI]
004013C8	. 84DB	TEST BL,BL
004013CA	~ 74 05	JE SHORT CRACKME.004013D1
004013CC	. 03FB	ADD EDI,EBX
004013CE	. 46	INC ESI
004013CF	^ EB F5	JMP SHORT CRACKME.004013C6
004013D1	> C3	RETN
004013D2	^ 2C 20	SUB AL, 20

EDI=000057BF
EAX=00000000

Then we keep tracing—this is the part where it evaluates our serial—I won't show it since I already know my serial is fake, and we arrive at this stage:

```

004013D8  33C0      XOR EAX,EAX
004013DA  33FF      XOR EDI,EDI
004013DC  33DB      XOR EBX,EBX
004013DE  8B7424 04  MOV ESI,DWORD PTR SS:[ESP+4]
004013E2  B0 0A     MOV AL,0A
004013E4  8A1E     MOV BL,BYTE PTR DS:[ESI]
004013E6  84DB     TEST BL,BL
004013E8  74 0B     JE SHORT CRACKME.004013F5
004013EA  80EB 30    SUB BL,30
004013ED  0FAFF8   IMUL EDI,EAX
004013F0  03FB     ADD EDI,EBX
004013F2  46       INC ESI
004013F3  EB ED     JMP SHORT CRACKME.004013E2
004013F5  81F7 34120000  XOR EDI,1234
004013FB  8BDF     MOV EBX,EDI
004013FD  C3       RETN
004013FE  FF25 84314000  JMP DWORD PTR DS:[&USER32.KillTimer]
00401404  FF25 88314000  JMP DWORD PTR DS:[&USER32.GetSystemMetrics]
0040140A  FF25 8C314000  JMP DWORD PTR DS:[&USER32.LoadCursorA]
00401410  FF25 90314000  JMP DWORD PTR DS:[&USER32.LoadAcceleratorTableA]
00401416  FF25 94314000  JMP DWORD PTR DS:[&USER32.MessageBeep]
0040141C  FF25 98314000  JMP DWORD PTR DS:[&USER32.GetWindowRect]
00401422  FF25 9C314000  JMP DWORD PTR DS:[&USER32.LoadStringA]
00401428  FF25 A0314000  JMP DWORD PTR DS:[&USER32.LoadIconA]
0040142E  FF25 A4314000  JMP DWORD PTR DS:[&USER32.LoadBitmapA]
EDI=00003039
Jump from 004013E8

Address Hex dump ASCII
0040218E 4B 41 52 41 53 55 00 00 KARASU...
00402196 00 00 00 00 00 00 00 00

```

There, basically what it does is XOR EDI, 1234—so that means that IF EDI = 57BF
 $\Rightarrow 57BF \text{ XOR } 1234 =$

Programador

57BF XOR 1234 =

458B

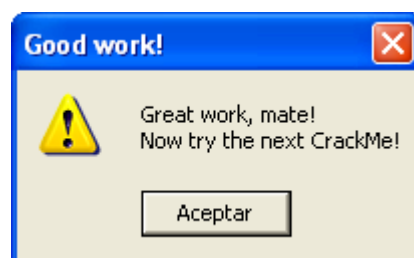
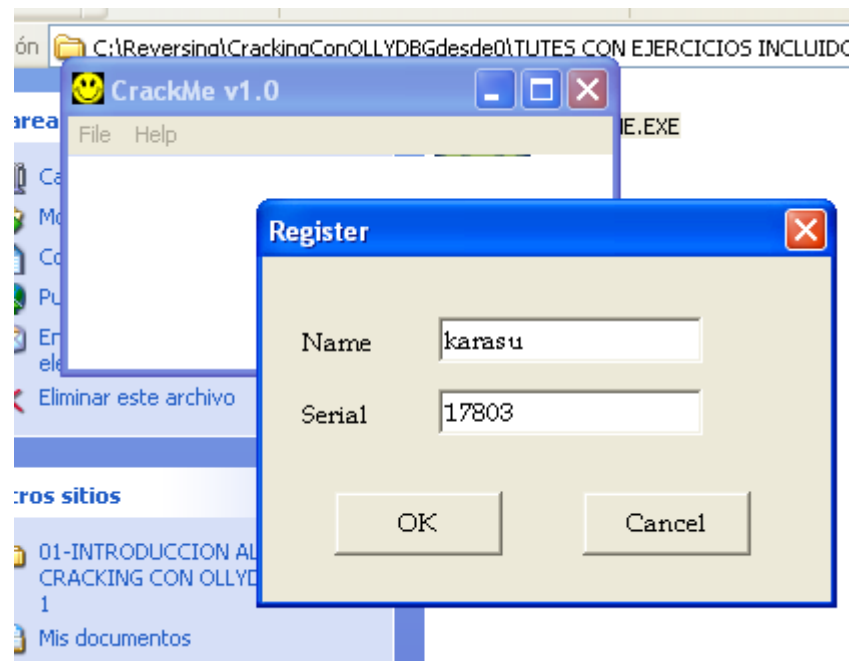
HEX

458B

DEC

17.803

Meaning that now it will be equal to 458B= which means that the serial would be 17803— the program continues, and that's when it throws the bad boy message—but we can test it



Upon clicking OK -> it returns the message 'Great work, mate! Now try the next Crackme!' - > so we beat it :D—something that might be worth noting is that the key 17803 works for both the string 'KARASU' and 'karasu'.

Alright, it's time to write a script.

```
MisProgramasEnPython > KeyGenCrackmeCrushead1999.py > ...
1  usu = input('Indique nombre de usuario: ')
2  usu = usu.upper()
3  key = 0
4  if usu.isalpha() == True:
5      for c in usu:
6          key = key + ord(c)
7      key = key^int(0x444C)
8      print('Su serial es: ' + str(key))
9  else:
10     print ("El usuario debe contener solo letras")
11
12
13
14
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS COMENTARIOS

```
PS D:\Eugenio\Propio\Desarrollo\MisProgramas> & D:/Eugenio/Propio/AreaTrabajo/Desarrollo/MisProgramas/MisProgramasEnPython/key.py
Indique nombre de usuario: karasu
Su serial es: 17803
PS D:\Eugenio\Propio\Desarrollo\MisProgramas>
```

Reviewing then, basically what needs to be done is for the script to ask me for a name (string) -> Then convert it to uppercase -> then add it up character by character -> then have it do XOR 5678, and apply XOR 1234 to that result, and finally return the numerical value.

Correction (08/15/2024) Someone pointed out to me that if you use, for example, 'Zapallo' as the username (just to make a quick mention—with 'zapallo' the previous keygen works without any issues), when generating the serial with that initial keygen—it wouldn't work and would trigger the bad boy message.

Because of this, I started investigating why something had slipped past me, and indeed it had: Upon tracing through it again, I realized that the problem is the Z.

	Hex dump	ASCII
E	5A 61 70 61 6C 6C 6F 00 00 00 00 00 00 00 00 00	Zapallo...
E	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Notice that 'Z' is 5A in Hex -> well, because there is a line that complicates things:

00401385	. 84C0	TEST AL,AL
00401387	. 74 13	JE SHORT CRACKME.0040139C
00401389	. 3C 41	CMP AL,41
0040138B	. 72 1F	JB SHORT CRACKME.004013AC
0040138D	. 3C 5A	CMP AL,5A
0040138F	. 73 03	JNB SHORT CRACKME.00401394
00401391	. 46	INC ESI
00401392	. ^EB EF	JMP SHORT CRACKME.00401383
00401394	> E8 39000000	CALL CRACKME.004013D2
00401399	. 46	INC ESI
0040139A	. ^EB E7	JMP SHORT CRACKME.00401383
0040139C	> 5E	POP ESI
0040139D	. E8 20000000	CALL CRACKME.004013C2
004013A2	. 81F7 78560000	XOR EDI,5678
004013A8	. 8BC7	MOV EAX,EDI
004013AA	. ^EB 15	JMP SHORT CRACKME.004013C1
004013AC	> 5E	POP ESI
004013AD	. 6A 30	PUSH 30
004013AF	. 68 60214000	PUSH CRACKME.00402160
004013B4	. 68 69214000	PUSH CRACKME.00402169
004013B9	. FF75 08	PUSH DWORD PTR SS:[EBP+8]
004013BC	. E8 79000000	CALL <JMP.&USER32.MessageBoxA>
004013C1	> C3	RETN
004013C2	. \$ 33FF	XOR EDI,EDI
004013C4	. 33DB	XOR EBX,EBX
004013C6	> 8A1E	MOV BL,BYTE PTR DS:[ESI]
AL=5A ('Z')		

There, when it compares and starts the loop to convert the characters to uppercase, etc.—'Z' (being 5A) being in AL—it is compared against 5A. This causes it, when it hits the instruction.

004013C4	. 33DB	XOR EBX,EBX	Registers (FP
004013C6	> 8A1E	MOV BL,BYTE PTR DS:[ESI]	EAX 0000003A
004013C8	. 84DB	TEST BL,BL	ECX 0012FDE4
004013CA	. 74 05	JE SHORT CRACKME.004013D1	EDX 7C91E4F4
004013CC	. 03FB	ADD EDI,EBX	EBX 00000000
004013CE	. 46	INC ESI	ESP 0012FE98
004013CF	. ^EB F5	JMP SHORT CRACKME.004013C6	EBP 0012FEB4
004013D1	> C3	RETN	ESI 0040218E
004013D2	\$ 2C 20	SUB AL,20	EDI 0012FF1C
004013D4	. 8806	MOV BYTE PTR DS:[ESI],AL	EIP 004013D4
004013D6	. C3	RETN	C 0 ES 0023
004013D7	. C3	RETN	P 1 CS 001B
004013D8	\$ 33C0	XOR EAX,EAX	A 0 SS 0023
004013DA	. 33FF	XOR EDI,EDI	Z 0 DS 0023
004013DC	. 33DB	XOR EBX,EBX	S 0 FS 003B
004013DE	. 8B7424 04	MOV ESI,DWORD PTR SS:[ESP+4]	T 0 GS 0000
004013E2	> B0 0A	MOV AL,0A	D 0
004013E4	. 8A1E	MOV BL,BYTE PTR DS:[ESI]	O 0 LastErr
004013E6	. 84DB	TEST BL,BL	EFL 00010206
004013E8	. 74 0B	JE SHORT CRACKME.004013F5	ST0 empty -??
004013EA	. 80EB 30	SUB BL,30	ST1 empty -??
004013ED	. 0FAFF8	IMUL EDI,EAX	ST2 empty -??
004013F0	. 03FB	ADD EDI,EBX	ST3 empty -??
004013F2	. 46	INC ESI	ST4 empty -NA
004013F3	. ^EB ED	JMP SHORT CRACKME.004013E2	ST5 empty 1.0
004013F5	> 81F7 34120000	XOR EDI,1234	ST6 empty 1.0
004013FB	. 8BDF	MOV EBX,EDI	ST7 empty 1.0
AL=3A (':')			
DS:[0040218E]=5A ('Z')			

Upon reaching SUB AL, 20-> it causes 5A - 20 = 3A, and that means this is represented... the output would look like this:

Address	Hex dump	ASCII
0218E	3A 41 50 41 4C 4C 4F 00 00 00 00 00 00 00 00 00	:APALLO.....
0219E	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

So the script needs to be modified because it doesn't take that into account; therefore, it is modified so that when a name starting with 'Z' is used -> it replaces it with ':' and performs the operations from there.

Using 'Zorro' as the name, I made the changes to the script and got these results:

```
MisProgramasEnPython > KeygenCrackmeCRUSHEAD1999.py > ...
1 import re
2 usu = input('Indique nombre de usuario: ')
3 permitido = r"[a-zA-Z0-9:]"
4 usu = usu.upper()
5 if re.match(f"^{permitido}+$", usu):
6     if usu[0] == 'Z':
7         usu = ':' + usu[1:]
8     key = 0
9     if usu:
10        for c in usu:
11            key = key + ord(c)
12            key = key^int(0x444C)
13        print('Su serial es: ' + str(key))
14    else:
15        print("El usuario debe contener solo letras")
16
17
18
19
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS COMENTARIOS

```
PS D:\Programas\Programacion\MisProgramas> & D:/Programas/Python/Python3
USHEAD1999.py
Indique nombre de usuario: Zorro
Su serial es: 17712
```

